Università della Calabria

Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e Sistemistica

---

Master Degree in ‘Robotics And Automation Engineering’

Course: Mobile Robotic Module 1

Planning, Control, Obstacle Avoidance and Localization of a Differential Drive vehicle

Student

Alessandro Furina
252328

Professor

Gianfranco Gagliardi

Academic Year 2024-2025

TABLE OF CONTENTS

INTRODUCTION

Nowadays, robotics is one of the most attractive sector for its wide field of applicability. Its applications are divided into two main categories: Industrial Robotic and Mobile Robotic. The difference between them are represented in the capability of motion of the robot. An industrial robot is typically fixed to a platform and, so, is mostly used for accuracy tasks in the factories like: pick and place objects, cut materials and so on. Often, they are composed of several arms linked by joints. These are fundamental to allow the robot to rotate. Its degrees of freedom are associated with the so named end effector that is the last arm of the robot responsible for performing the task required. On the contrary, a mobile robot can move in the space thanks to a system of locomotion represented by wheels (for terrier applications) or propellers (for aerial applications). This means that also the presence of random obstacles must be taken into account to avoid unexpected collisions. Based on the ability of the robot to represent the surroundings and the reactivity to properly take decisions, mobile robots are described by different levels of autonomy. In this project the aim consists of guiding a terrestrial mobile robot from a start to an end point in a closed environment (it could be a room for instance) in presence of fixed and known obstacles. This means that the robot has knowledge of the position of each obstacle through a map of the environment. The typical structure to sort off this problem can be represented as follows:

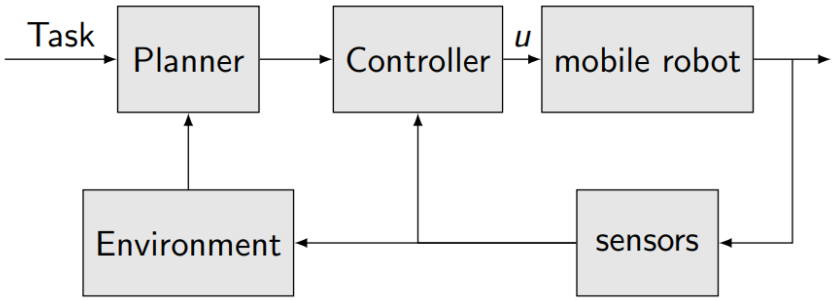

Figure: Mobile robot control scheme

Where:

- The planner module receives the task (start point, end point and eventually features regarding maximum time to complete the path, energy consumption and similar). It is responsible for designing the path guaranteeing obstacle avoidance.
- The controller module generates the input commands to steer the robot.
- The mobile robot module incorporates the dynamic of the vehicle, function of its physical structure and geometry.
- The sensors, if enabled, are useful to acquire information regarding the local surrounding allowing better and safer path following.
- The environment contains all the information regarding the position of the obstacles, walls and so on.

1. MOBILE ROBOT

Mobile robots mainly differ each other for their geometry. This can include for instance the number of the wheels, their characteristics in terms of structure and location, position of the center of gravity and so on. Based on this information each robot can be described by a set of equations of dynamic. In this set the most important regard the description of the motion of the center of gravity in terms of velocity along the two axes and the orientation of the robot with respect to the x-axis. In this project a differential drive vehicle was selected for its simplicity in terms of dynamic. It can be represented as follows:

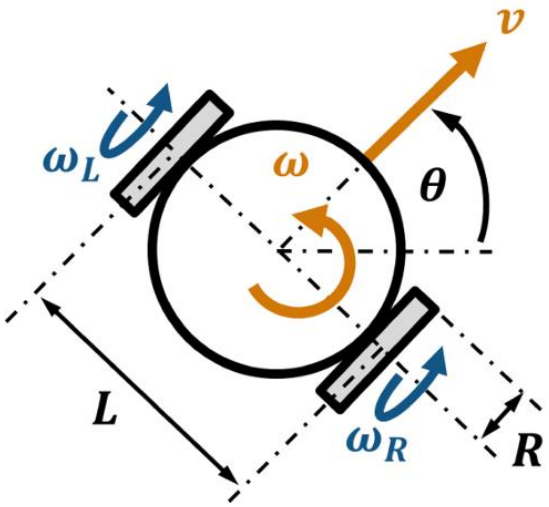

Fig. 1.1: Differential Drive scheme

With:

- $R = 15\ [cm]$ radius of the wheel (equal for both).
- $L = 50\ [cm]$ wheelbase of the vehicle. It represents the distance between the wheels.
- w_l and w_r represent the left and right angular speed of the two wheels. They are the input for the vehicle.
- v is the linear speed of the entire vehicle.
- w is the angular speed of the entire vehicle. Together with v is an output for the vehicle.
- θ is the angle between the orientation of the vehicle and the x-axis.

Since w_l and w_r can be different, the robot is able to steer following circular trajectories. Moreover, if one of them is zero, it can rotate without moving (linear speed equal to zero). Based on the knowledge of w_l and w_r rather than v and w and vice versa we can distinguish between:

FORWARD KINEAMATICS

Given the input of the model compute the output:

$$\begin{cases} v = \frac{R}{2} * (w_r + w_l) \\ w = \frac{R}{L} * (w_r - w_l) \end{cases} \quad \textbf{(1.1)}$$

INVERSE KINEAMATICS

Given the output of the model compute the input:

$$\begin{cases} w_l = \frac{1}{R} * (v - \frac{wL}{2}) \\ w_r = \frac{1}{R} * (v + \frac{wL}{2}) \end{cases} \quad \textbf{(1.2)}$$

The following code creates the vehicle in Matlab:

```
                        %% MOBILE ROBOT
% Define the model of the vehicle:
r=0.15; %wheel radius
b=0.5; %wheelbase
vehicle=DifferentialDrive(r,b);
```

2. ENVIRONMENT MODULE

In this section all the information regarding the surrounding is stored. This includes start and end points, position and structure of obstacles and walls. As already mentioned, the map of the environment is available so that the a priori information is complete. The following code lines are responsible to initialize the settings:

```
                    %% ENVIRONMENT
% Creation of the environment
map=im2bw(imread("map.bmp"));
map(1,:)=false; %upper wall
map(1:end,1)=false; %left wall
map(end,:)=false; %lower wall
map(1:end,end)=false; %right wall
start=[25 792]; %expressed according to the representation (row,col)
goal=[535 10];%expressed according to the representation (row,col)
```

The map appears as follows:

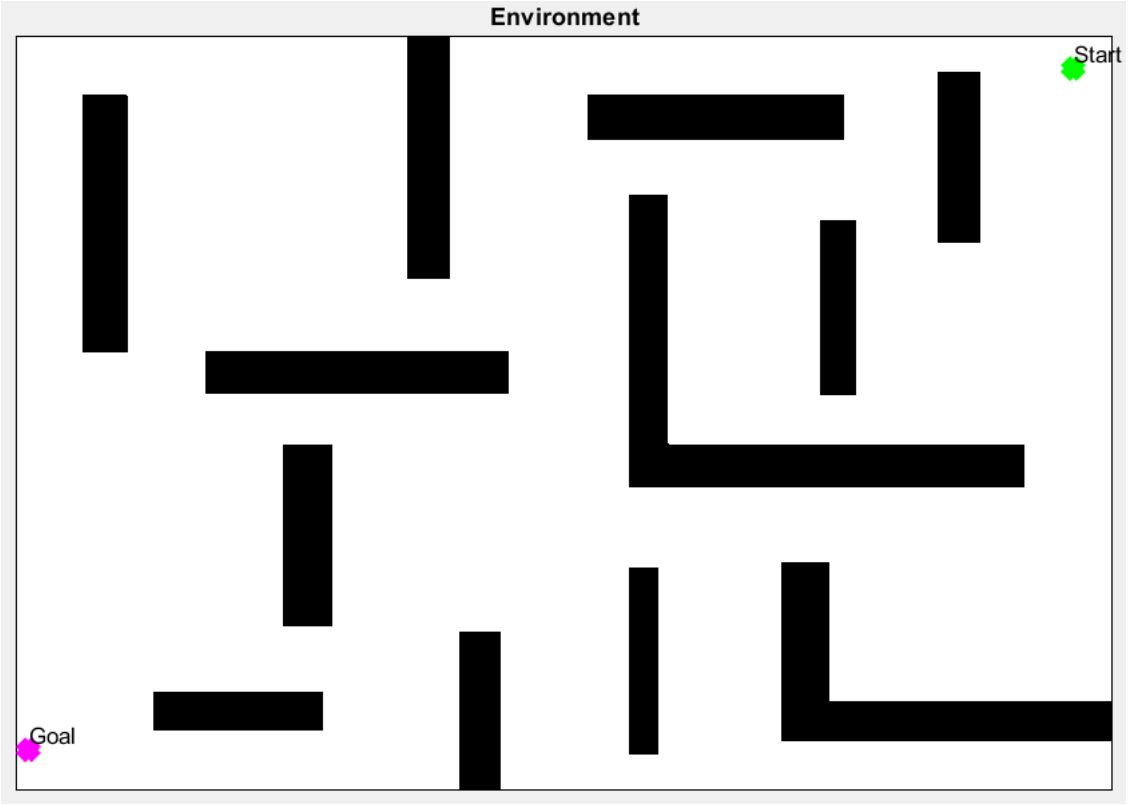

Fig. 2.1: Environment

It is an 821[cm] x 565 [cm] environment.

3. PLANNER MODULE

As previously introduced, the planner module is responsible for designing the trajectory from the start to the end point ensuring that the robot never collides with obstacles. It's important to remark that these are fixed and well-known so that, once the trajectory is computed, it is sufficient to apply a good path following strategy to complete the task without facing with unexpected issues. To sort off the path planning problem, there exist several techniques and the choice of one rather than the other is related to the features the designer wants to achieve. In this application the path planning strategy chosen is the Artificial Potential Field (APF). It was inspired by looking at the behaviour of a mass in a gravitational field or of a point charge in an electromagnetic field. The main idea is to associate an attraction field able to guide the robot towards the goal. This means that the scalar field must have a minimum (global if possible) in the goal position. In order to avoid collision with obstacles during the path, a repulsive field is added. In this case the aim consists of having repulsive forces arising from the obstacles and acting on the robot. Thus, the repulsive scalar field must have maximum in the occupied position. Putting together the two characteristics we end up with the following function:

$$J(x,y) = w_a * J_a\left(x, y, x_{goal}, y_{goal}\right) + w_r * J_r(x, y, x_{obstacle_i}, y_{obstacle_i}) \qquad \textbf{(3.1)}$$

Where:

- $J(x,y)$ is the total scalar field.
- $w_a * J_a\left(x, y, x_{goal}, y_{goal}\right)$ is the attractive scalar field weighted by a scalar w_a.
- $w_r * J_r(x, y, x_{obstacle_i}, y_{obstacle_i})$ is the repulsive field weighted by a scalar w_r where:

$$i = 1, \dots, \#obstacles$$

The choice of the scalar functions J_a and J_r is up to the designer based on considerations about the environment in which the robot is moving. In this project the following choices were adopted:

Attractive Field:

$$J_a = \sqrt{(x - x_{goal})^2 + (y - y_{goal})^2} \qquad \textbf{(3.2)}$$

That is the Euclidean Norm between each point and the goal. This function ensures a minimum (global since the function is quadratic) set in the goal position. The attractive component is computed by the following function:

```
function [z] = attractiveField(x,y,goal)
    z=sqrt((x-goal(2)).^2+(y-goal(1)).^2);
end
```

Applied to our case, the result is the following:

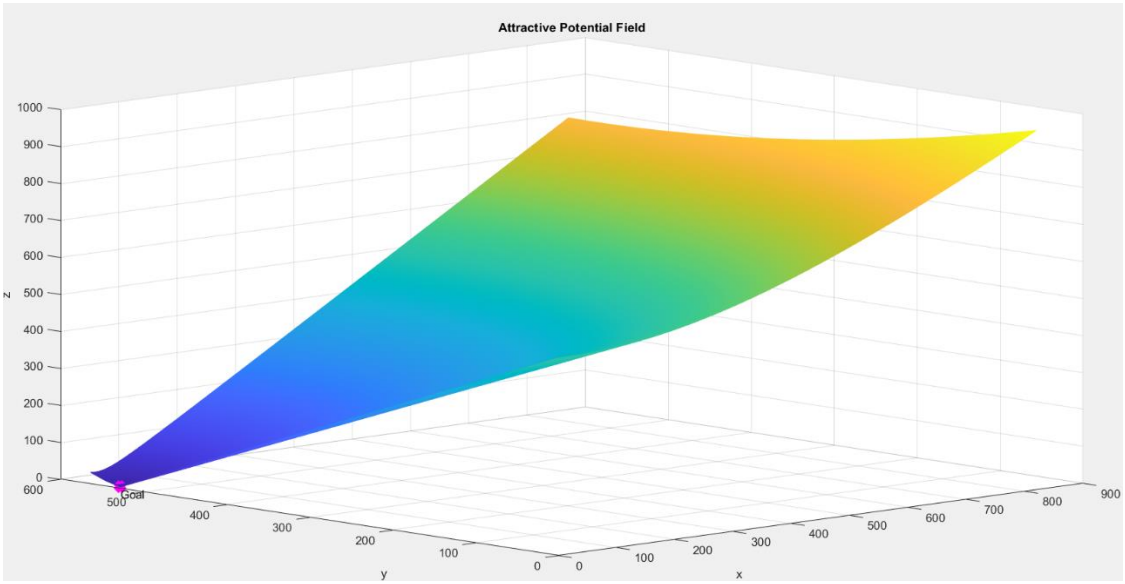

Fig 3.1: Attractive Potential Field

Repulsive Field:

$$J_r = \begin{cases} \frac{k_r}{\gamma} * \left(\frac{1}{d_i(q)} - \frac{1}{d_{o,i}}\right)^{\gamma} & if\ d_i(q) \leq d_{o,i} \\ 0 \quad otherwise \end{cases} \qquad \textbf{(3.3)}$$

Where:

- k_r and γ are scalar tuning parameters.
- $d_i(q)$ is the minimum distance (in Euclidean sense) between the current position of the robot and the i-th component of the obstacle. They are considered as a set of convex components, each one associated with the repulsive force.
- $d_{o,i}$ is the threshold determining the influence of each obstacle in the repulsive field. A small value implies a close passage of the robot near the obstacle and vice versa.

The repulsive field is computed by the following function:

```
function [z] = repulsiveField(map)
    min_distances=bwdist(~map)+1;
    gamma=2;
    kr=500;
    d0=30;
    z=(kr/gamma)*((1./min_distances)-(1/d0)).^gamma;
    for i=1:size(z,1)
        for j=1:size(z,2)
            if(min_distances(i,j)>d0)
                z(i,j)=0;
            end
        end
    end
end
```

Since the shape of the obstacles has been considered with its real geometry, without extra-filling operation, for safety reasons $d_{o,i}$ is set to a large value. In this way we ensure that the robot never touches the border of any obstacle.

The repulsive field appears as follows:

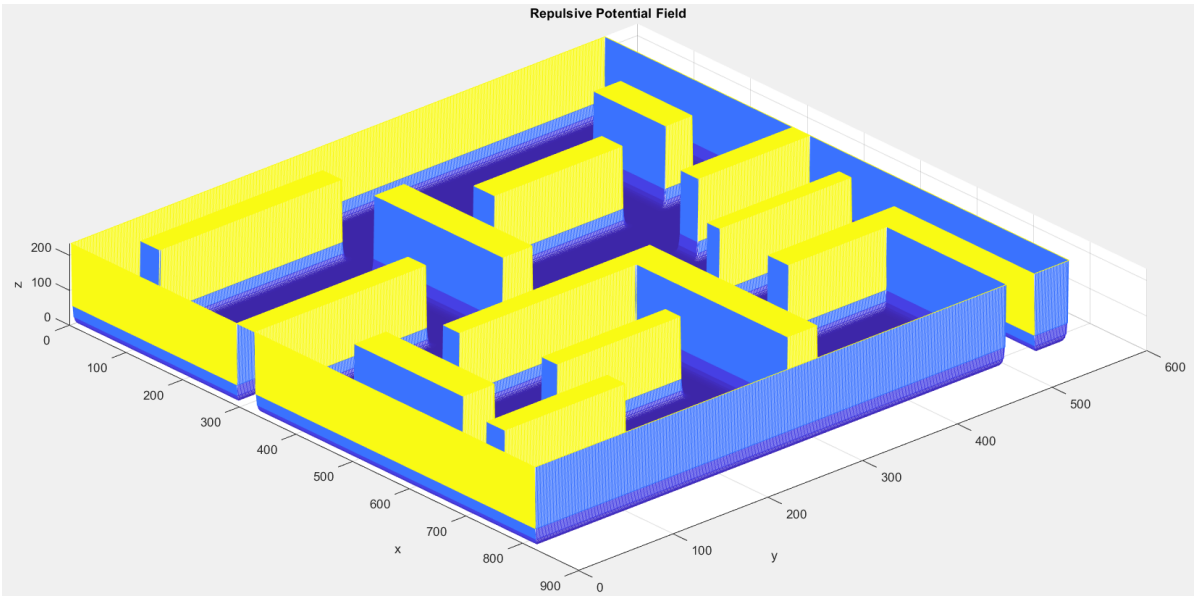

Fig. 3.2: Repulsive Potential Field

By selecting $w_a = 10$ and $w_r = 20$, the total field is the following:

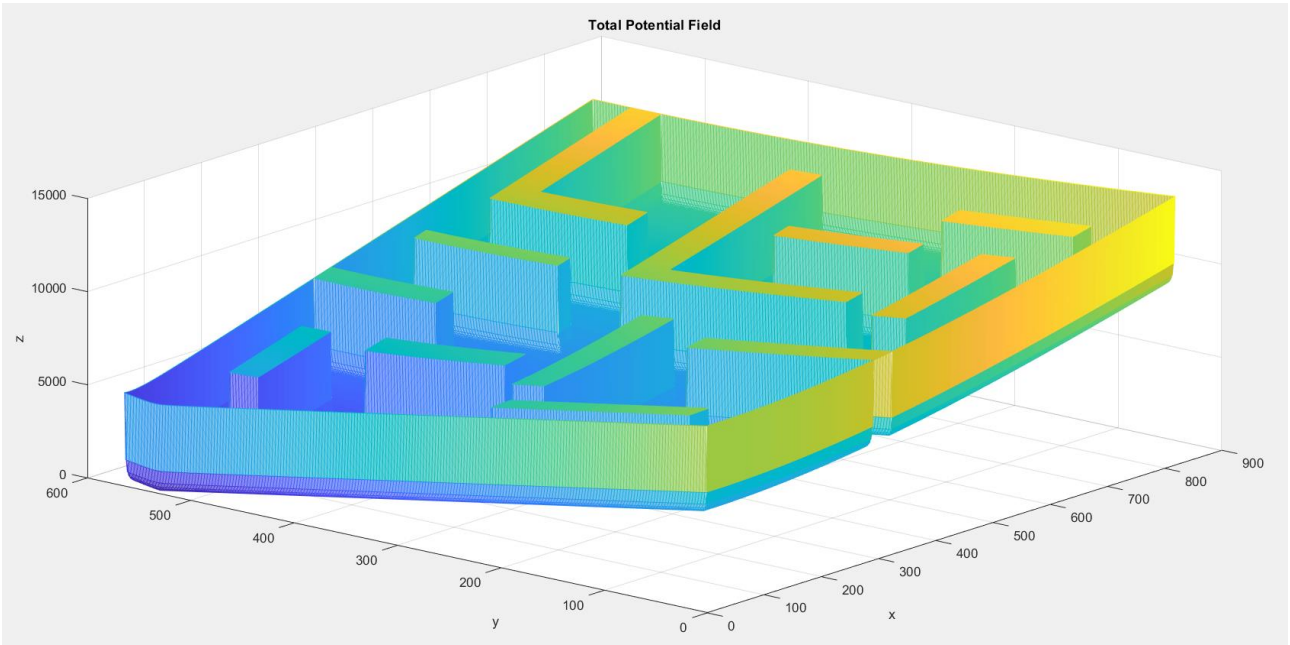

Fig. 3.3: Total Potential Field

Once the scalar field is computed, the robot now needs to be guided in this artificial environment. The theory of the APF technique suggests to recall the basic property of the gradient vector. Indeed, it is well known that the gradient vector computed in a generic *(x,y)* points toward the maximum of the function. This means that the anti-gradient direction is

pointing the minimum. Thus, starting from the initial position, the robot must follow the anti-gradient direction (computed for each point) to reach the minimum, so the goal. This represents a powerful strategy in mobile robotic, but it does not ensure always convergence to the minimum because, despite the attractive component is quadratic, the function that comes out by adding the repulsive part could not be quadratic yet. This means that local minimum may arise having as a consequence that the robot gets stuck even if it has not reached the goal. To sort off this problem there are several ways, one of these is represented by the vortex potential field which adds to the repulsive part a vortex component on each obstacle to allow the robot to pass around it. The following code is responsible for computing the anti-gradient direction for each scalar field introducing a vortex component:

```
                    %% PLANNER
%Attractive Potential Field:
[x,y]=meshgrid(1:size(map,2),1:size(map,1));
z_attractive=attractiveField(x,y,goal);
[dxa,dya]=gradient(z_attractive);
%Repulsive Potential Field:
z_repulsive=repulsiveField(map);
[dxr,dyr]=gradient(z_repulsive);
dxr_tmp=dxr;dyr_tmp=dyr;%useful to introduce the vortex field
dxr=dxr+dyr_tmp;dyr=dyr-dxr_tmp;%vortex field
%Total Potential Field:
z_total=10*z_attractive+20*z_repulsive;
dx=10*dxa+20*dxr;
dy=10*dya+20*dyr;
dx_norm=dx./sqrt(dx.^2+dy.^2);%normalized
dy_norm=dy./sqrt(dx.^2+dy.^2);%normalized
```

The anti-gradient related to the attractive component appears as follows:

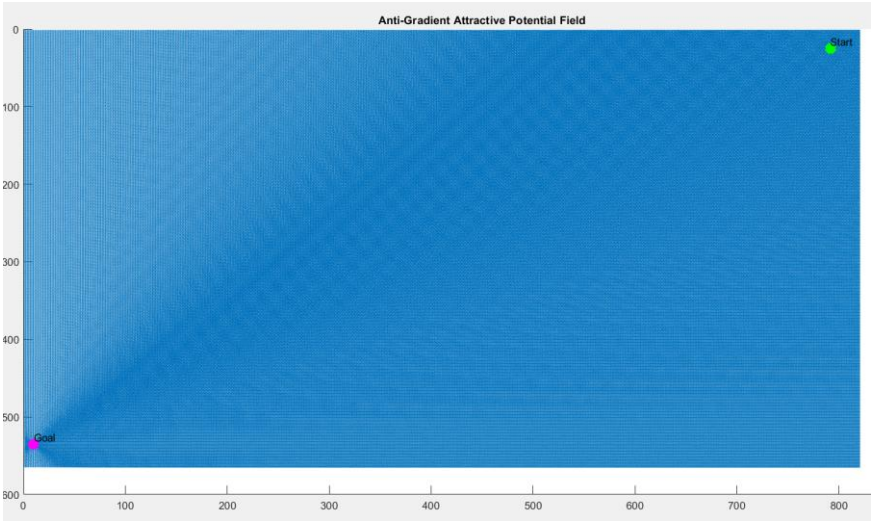

Fig. 3.4: Anti-Gradient Attractive Potential Field

With focus on the goal:

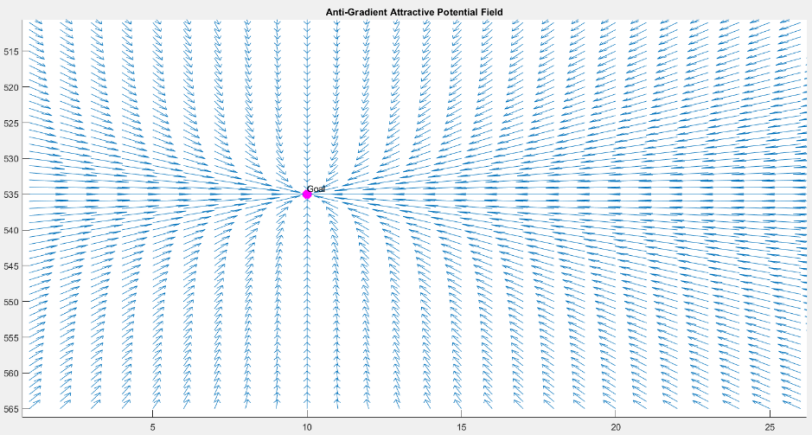

Fig. 3.5: Anti-Gradient Attractive Potential Field (Goal zoom)

While for what concerns the repulsive part:

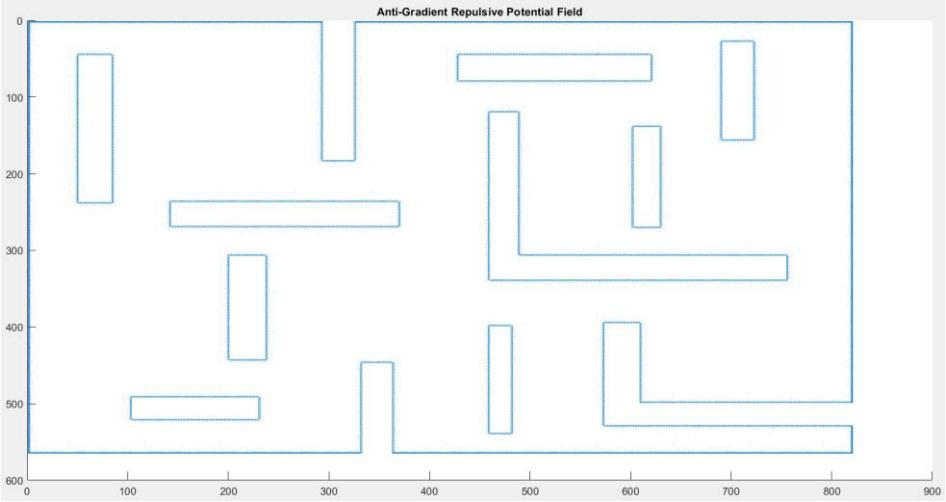

Fig. 3.6: Anti-Gradient Repulsive Potential Field

With a focus on the upper right obstacle:

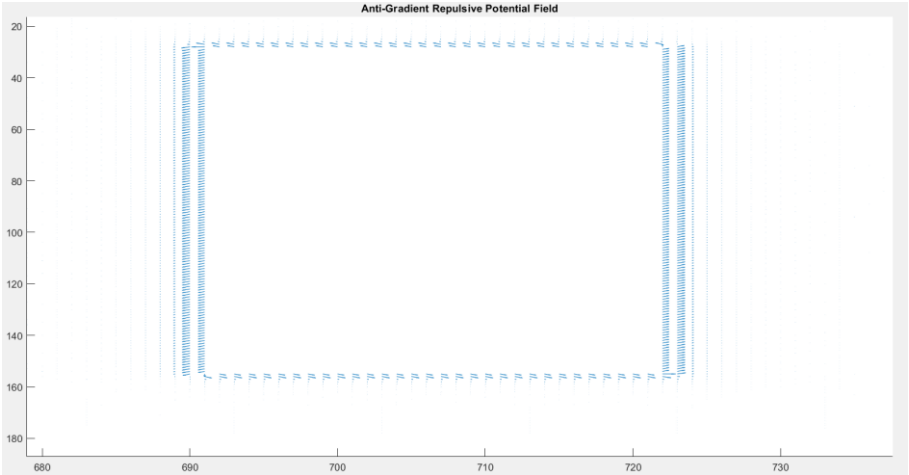

Fig. 3.7: Anti-Gradient Repulsive Potential Field (zoom)

Finally, the total anti-gradient is:

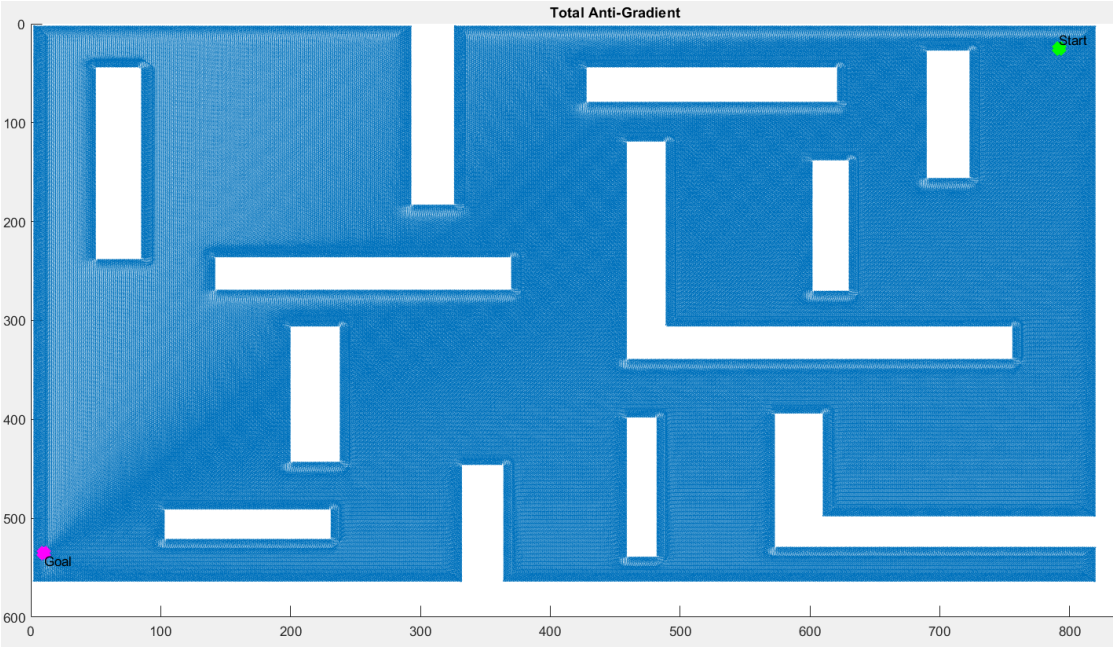

Fig. 3.8: Total Anti-Gradient

With zoom on the goal:

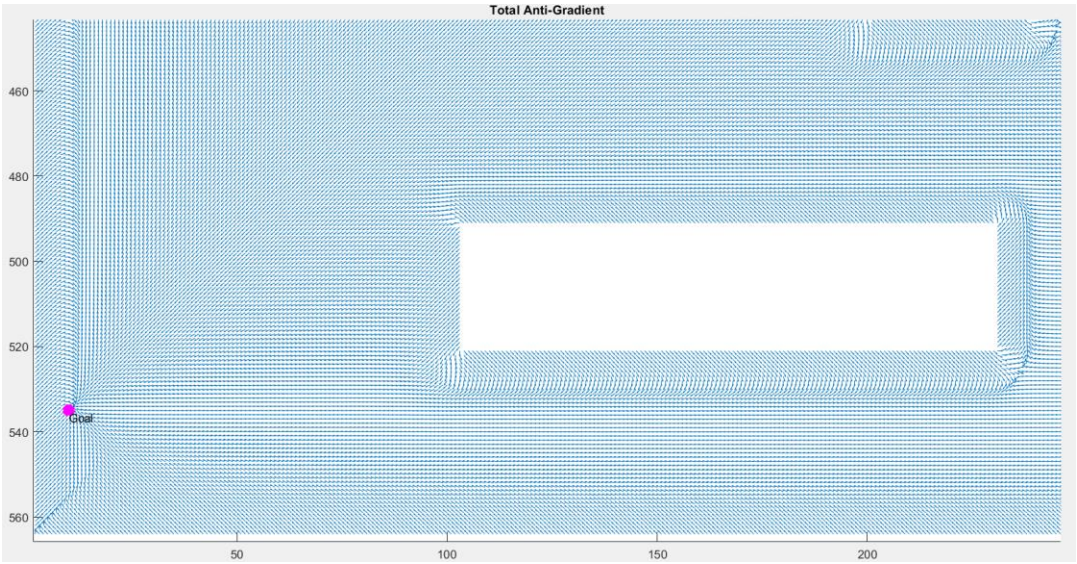

Fig. 3.9: Total Anti-Gradient (Goal zoom)

Now, in order to obtain the trajectory, the following plan function allows to apply the descendant method for each pose of the robot:

```
function [x_traj,y_traj] = plan(start,goal,dx,dy)
    maxIter=50000;
    treshold=1;
    alpha=0.1;
    trajectory=start;
    for i=2:maxIter
        if(norm(trajectory(i-1,:)-goal)<treshold)
            break;
        end
        last_position=trajectory(i-1,:);
        current_dx=dx(round(last_position(1)),round(last_position(2)));
        current_dy=dy(round(last_position(1)),round(last_position(2)));
        trajectory(i,:)=last_position+alpha*([current_dy,current_dx]);
    end
    x_traj=trajectory(:,2);% x coordinate is related to the col
    y_traj=trajectory(:,1);% y coordinate is related to the row
end
```

The trajectory is completed when the robot is 1 [cm] far (in Euclidean sense) from the goal.

Giving to this function the start, the goal and the normalized total anti-gradient the trajectory computed by the planner appears as follows:

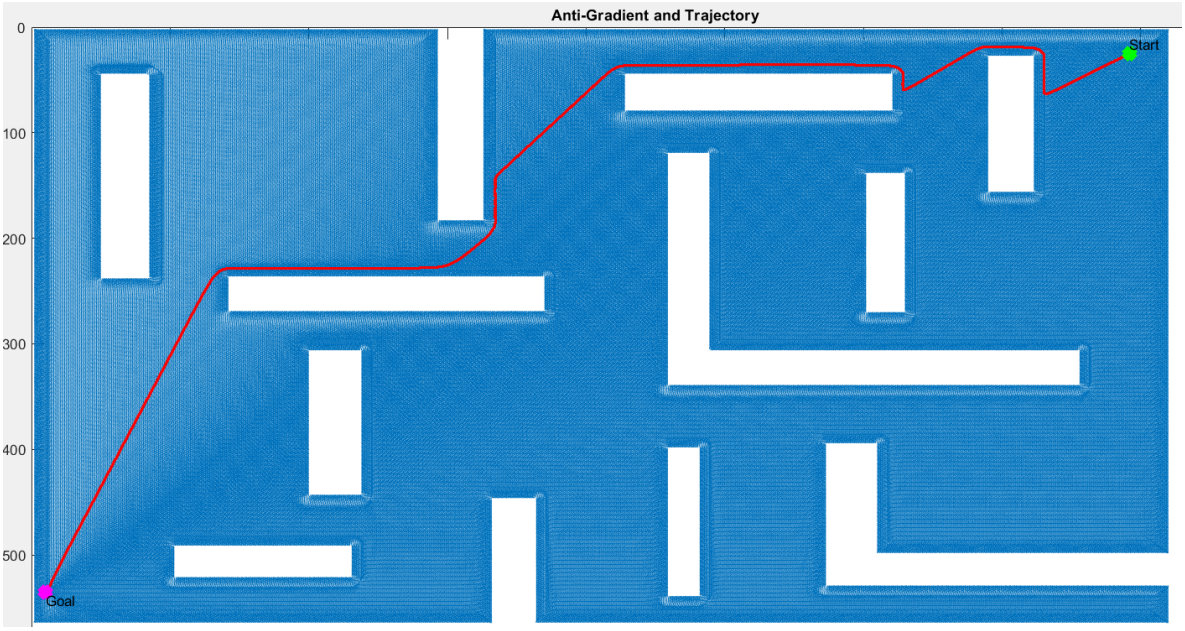

Fig. 3.10: Total Anti-Gradient and trajectory

It's possible to appreciate that the robot maintains a safe distance from the obstacles thanks to the tuning of the parameters in the repulsive part.

4. CONTROLLER MODULE

The controller module is responsible for implementing a path following strategy. Indeed, once the trajectory is available thanks to the Planner Module, the robot needs to be driven along the trajectory. This means that at each time instant the controller must design the angular speeds of the wheels in order to make the differential drive capable to track the path. To solve this problem the Pure Pursuit strategy was selected. It was proposed to control vehicles described by bicycle model as shown in the figure below:

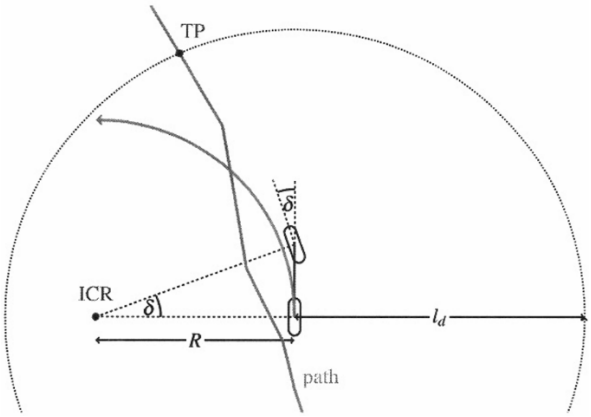

Fig. 4.1: Pure Pursuit representation

The strategy consists of identifying a look ahead distance l_d to select a target point TP away from the vehicle. At each time instant the Pure Pursuit Controller modifies, if necessary, the front steering angle δ to make the vehicle capable of reaching the target point TP. By selecting the target points from the trajectory, repeating this strategy in time the robot should be able to follow the path. A big value of l_d implies smoother tracking path but with less accuracy. On the contrary, a small value of l_d can generate instability since the controller tries to be as much accurate as possible:

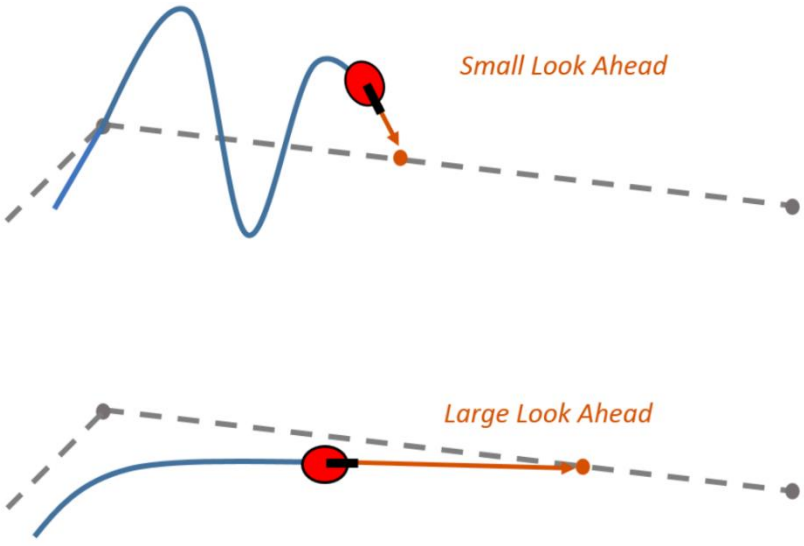

Fig. 4.2: Small vs Large Look Ahead Distance

Where the grey dotted line represents the trajectory to be tracked.

In the following code lines the Pure Pursuit Controller of Matlab is initialized together with some variables referring to the sampling time and the current pose of the robot:

```
                %% CONTROLLER
initial_pose=[start(2) start(1) 0];% reminder:start is expressed as (row,col)
% Define the controller:
controller=controllerPurePursuit;
controller.Waypoints=trajectory;
controller.LookaheadDistance=0.5;
controller.MaxAngularVelocity=1.5;
controller.DesiredLinearVelocity=1;
maxIter=2*length(trajectory);
treshold=1;
Ts=0.5;
pose(1,:)=initial_pose;
i=2;
input_commands=[0 0];
```

The control action is implemented by a while loop in which the new pose is obtained by combining the forward and inverse kinematics:

```
% Simulation Loop:
while i<=maxIter
    [v_ref,w_ref]=controller(pose(i-1,:));% reference speeds
    [wl,wr]=vehicle.inverseKinematics(v_ref,w_ref);% left and right angular speeds
    [v,w]=vehicle.forwardKinematics(wl,wr);% linear and angular speeds
    velb=[v; 0; w];
    vel=bodyToWorld(velb,pose(i-1,:));
    pose(i,:)=pose(i-1,:)+vel'*Ts;
    input_commands(i,:)=[v,w];
    if(norm(pose(i,1:2)-[goal(2) goal(1)])<treshold)
            break;
    end
    i=i+1;
end
```

The robot is controlled until it is 1 [cm] far from the goal. The controlled pose is the following:

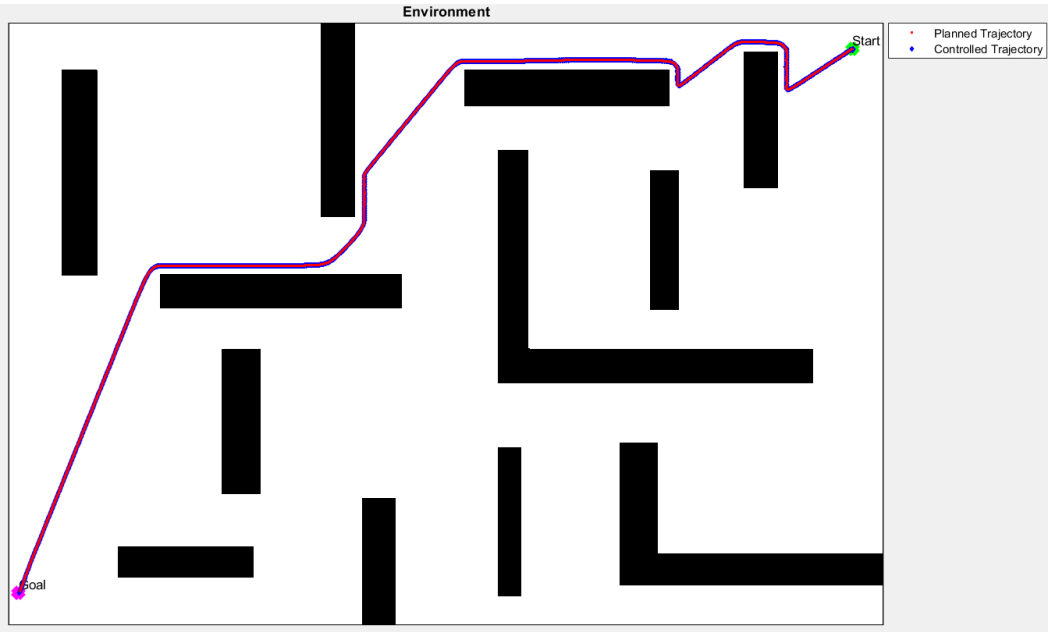

Fig. 4.3: Planned and Controlled trajectories

It's possible to notice that the robot successfully tracked the path.

5. OBSTACLE AVOIDANCE

Let's move on with a problem which involves also the Sensor Module. The task we want to address consists of guiding the robot from the start to the end point avoiding the obstacles. The difference with respect to the previous case is that, instead of tracking the path computed from the planner, in this case some waypoints are added in the environment so that the vehicle must pass through. However, doing this operation the robot could be faced with some obstacles and must be able to avoid them. To solve the tracking problem the previous strategy is used, that is the Pure Pursuit Control design. For what concerns the obstacle avoidance, the vehicle is equipped with Lidar sensors to obtain measurements of the nearest obstacles and the Vectorial Histogram Field (VFH+) algorithm is used to avoid collisions. It consists of 3 phases:

1. A 2D Cartesian histogram grid is used to represent obstacles. Each cell in the grid has a certainty value representing the confidence that an obstacle resides in that cell. A cell is updated when the sensor's readings detect again that the cell is occupied. At each time instant only the cells belonging to the so called "Active Window" are updated. It represents an area around the robot centre. Finally, based on the histogram grid, a polar grid is created as shown in figure:

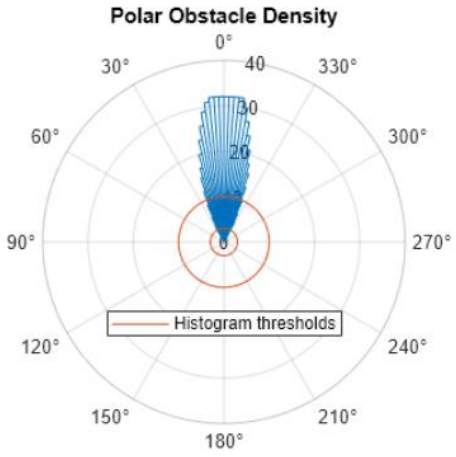

Fig. 5.1: Polar Histogram Grid

   The sectors of the graph are numerically evaluated considering the active window's values.

2. The polar grid is converted into a binary polar histogram grid. The process takes into account the values of each sector compared to the minimum threshold. Sectors with values greater than the minimum threshold are considered occupied, free otherwise.
3. The binary polar histogram grid is converted into a masked polar histogram on which it's possible to compute the steering direction to avoid the occupied sectors. It is calculated by optimizing a cost function whose parameters are tunable:

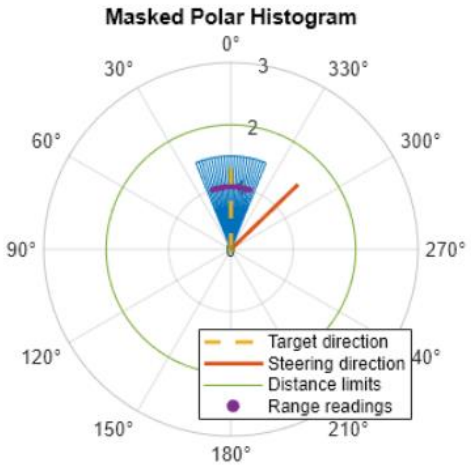

Fig. 5.1 :Masked Polar Histogram Grid

Let's have a look at the implementation:

```
%% OBSTACLE AVOIDANCE
% Create the controller VFH+:
controller_VFH=controllerVFH;
% Set the minimum and maximum distance of an obstacle to be detected:
controller_VFH.DistanceLimits=[1 50];
% Set the number of sectors around the robot:
controller_VFH.NumAngularSectors = 180;
% Set the treshold:
controller_VFH.HistogramThresholds = [5 10];
controller_VFH.RobotRadius = r;
controller_VFH.SafetyDistance = 2*r;
controller_VFH.MinTurningRadius = 0.25;
% Define the pose:
visualizer_map=occupancyMap(~map);
pose_VFH=zeros(3,length(input_commands));
pose_VFH(:,1)=[initial_pose(1) visualizer_map.DataSize(1)-initial_pose(2) initial_pose(3)];
% Define the Lidar sensor:
lidar = LidarSensor;
lidar.sensorOffset = [0,0];
lidar.scanAngles = linspace(-2*pi,2*pi,51);
lidar.maxRange = 50;
lidar.mapName=visualizer_map;
% Define the visualizer:
visualizer=Visualizer2D;
visualizer.hasWaypoints=true;
visualizer.mapName='visualizer_map';
visualizer.attachLidarSensor(lidar);
```

```
% Define the waypoints:
waypoints=[start(2) visualizer_map.DataSize(1)-start(1);
    780 500;
    800 400;
    800 250;
    700 150;
    600 200;
    344 170;
    200 20;
    goal(2) visualizer_map.DataSize(1)-goal(1)];
% Define the Pure Pursuit controller:
controller.release;
controller.Waypoints=waypoints;
controller.LookaheadDistance=2;
controller.MaxAngularVelocity=0.5;
controller.DesiredLinearVelocity=1;
```

The above code is responsible for settings all the objects useful for the simulation such as: the VFH Controller, the Lidar, the Visualizer, the Pure Pursuit Controller and the waypoints to be tracked. The obstacle avoidance operation is realized by the following loop:

```
i=2;maxIter_VFH=10000;
danger=false;
while(i<maxIter_VFH)
    if(norm(pose_VFH(1:2,i-1)-[goal(2) visualizer_map.DataSize(1)-goal(1)]')<treshold)
            break;
    end
    [v_ref,w_ref,lookAheadPoint]=controller(pose_VFH(:,i-1));% reference speeds
    target_direction = atan2(lookAheadPoint(2)-(visualizer_map.DataSize(1)-pose_VFH(2,i-1))...
      ,lookAheadPoint(1)-pose_VFH(1,i-1))-pose_VFH(3,i-1);
    readings=lidar(pose_VFH(:,i-1));
    steering_direction=controller_VFH(readings,lidar.scanAngles,target_direction);
    steering_direction=mod(steering_direction+pi,2*pi)-pi;
    target_direction=mod(target_direction+pi,2*pi)-pi;
    danger=any(readings<2*controller_VFH.SafetyDistance);
```

```
    if danger
        controller_VFH.PreviousDirectionWeight=3;
        w_ref_danger=0.3*steering_direction;
        v_ref_danger=0.1;
        velb=[v_ref_danger; 0; w_ref_danger];
    else
         velb=[v_ref; 0; w_ref];
    end
    vel=bodyToWorld(velb,pose_VFH(:,i-1));
    pose_VFH(:,i)=pose_VFH(:,i-1)+vel*Ts;
    pause(0.0001);
    visualizer(pose_VFH(:,i),waypoints,readings)
    i=i+1;
end
```

At each iteration, the Pure Pursuit Controller computes the reference speeds to reach the target points. Moreover, Lidar readings allow the Controller VFH to calculate the steering and target directions. If some reading results to be lower than two times the safety distance, the *danger* variable becomes true to express that a danger situation is detected. In this case, a new angular speed is computed based on the steering direction while the linear speed reduces in order to guarantee a safe manoeuvre. Otherwise, the evolution proceeds based on the standard Pure Pursuit Controller. The final evolution appears as follows:

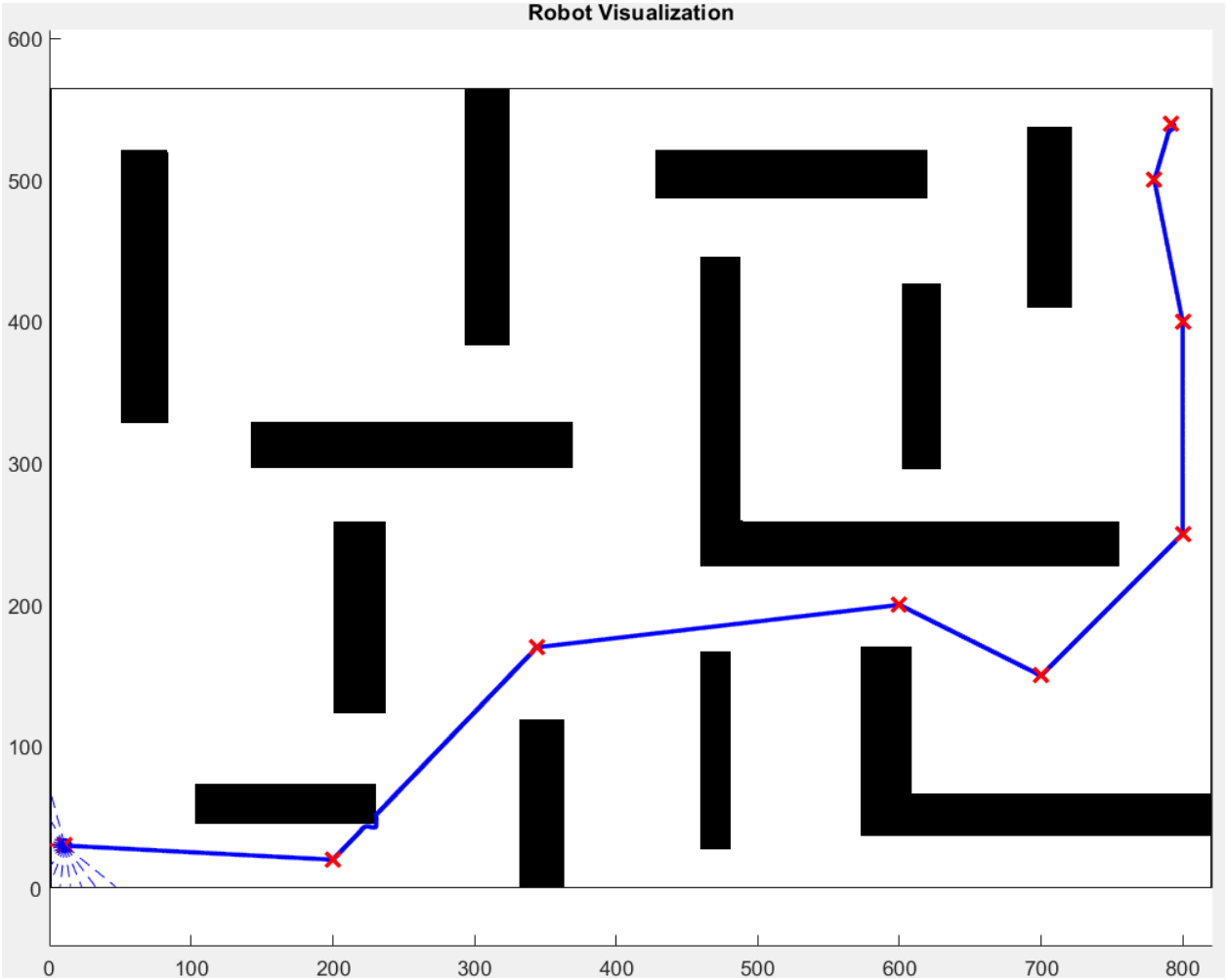

Fig. 5.2 : Evolution of the robot trajectory

We can notice that the robot successfully reached the goal marking all the waypoints. In particular:

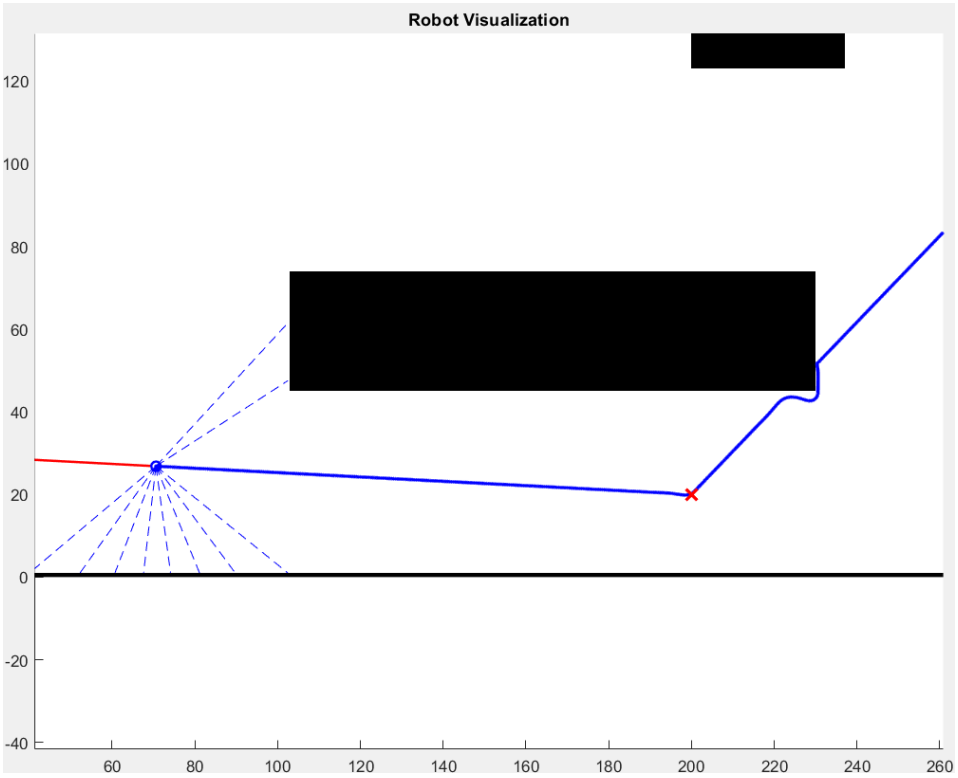

Fig. 5.3 : Snapshot of the robot trajectory

The robot was able to avoid the collision passing around the obstacle.

6. LOCALIZATION

Last part of this project is oriented to sort off the localization problem. So far, we succeeded to plan a trajectory and to track it. Moreover, the obstacle avoidance problem was solved by introducing sensors readings in order to react to critical directions. Now, we want to focus on the real-time localization of the robot. Indeed, it is useful to have information about the current position and orientation of the robot while moving in the environment. The problem we are addressing is based on the knowledge of the surrounding from the environment module. To face this new scenario, it's necessary to resort the Kalman Filter theory. It finds a huge field of applications in several sectors since it allows to estimate the value of a variable in a noisy framework. Most of the time noisy signals arise due to the conditions in which the system is working in or, basically, due to inaccuracy of the sensors used. Thus, considering the value of the state as it is described by the devices would be a bad strategy. Moreover, it is not always possible to have a direct access to these measurements. In these cases, the state remains "confined" as an internal variable whose value can be estimated through a filtering procedure on the output that by definition is a known signal of the system. In the context of interest, the unknown variable is the state while the known one is the output. Both are random since they are affected respectively by the process and measurement noise. The model of the system is the following:

$$\begin{cases} x_1(k+1) = x_1(k) + v * \cos\big(x_3(k)\big) * T_s + v_1(k) \\ x_2(k+1) = x_2(k) + v * \sin\big(x_3(k)\big) * T_s + v_1(k) \\ x_3(k+1) = x_3(k) + w * T_s + v_1(k) \\ z(k) = \sqrt{(x_i - x_1(k))^2 + (y_i - x_2(k))^2} + w(k) \\ \varphi(k) = arctg\left[\frac{y_i - x_2(k)}{x_i - x_1(k)}\right] - x_3(k) + w(k) \end{cases} \qquad \textbf{(6.1)}$$

With $i = 1, \dots, \#sensor\ readings$.

Where:

- The state of the system is:

$$\begin{bmatrix} x_1(k) \\ x_2(k) \\ x_3(k) \end{bmatrix} = \begin{bmatrix} x_{CG} \\ y_{CG} \\ \theta \end{bmatrix}$$

- v and w are respectively the linear and angular speed of the robot.
- T_s is the sampling time.
- $v_1(k)$ and $w(k)$ are respectively the process and the measurement noise. They are described by a Gaussian distribution of zero mean.
- $z(k)$ is the output associated with the sensor reading of the point (x_i, y_i). It represents the Euclidean norm with respect to the first and second state component.

- $\varphi(k)$ is the output related to the bearing angle between the orientation of the robot and the point detected.

Kalman theory finds its maximum expression with the so known Kalman Filter. It represents a very powerful algorithm capable to provide at each time instant both an estimation and a prediction of the state and of the output starting from the knowledge of the output. However, Kalman Filter operates only on Discrete-time Linear system. Thus, since the robot description is non-linear both in the state evolution and in the output, in order to adopt this approach for our system we need to resort the alternative version of the Kalman Filter, known as Extended Kalman Filter (EKF). Starting from a generic representation of the system:

$$\begin{cases} x_{k+1} = f(x_k, u_k) \\ y_k = g(x_k, u_k) \\ x_0 = x0 \end{cases} \qquad \textbf{(6.2)}$$

The EKF evolution is:

$$\begin{cases} C_k = \frac{\partial g}{\partial x_k} \mid \hat{x}_{k|k-1}, u_k \\ L_k = P_{k|k-1} * C_k^T * (C_k * P_{k|k-1} * C_k^T + V)^{-1} \\ \hat{y}_{k|k-1} = g(\hat{x}_{k|k-1}, u_k) \\ \hat{x}_{k|k} = \hat{x}_{k|k-1} + L_k * (y_k - \hat{y}_{k|k-1}) \\ P_{k|k} = P_{k|k-1} - L_k * C_k * P_{k|k-1} \\ \hat{x}_{k+1|k} = f(\hat{x}_{k|k}, u_k) \\ A_k = \frac{\partial f}{\partial x_k} \mid \hat{x}_{k|k}, u_k \\ P_{k+1|k} = A_k * P_{k|k} * A_k^T + W \\ Given\ \hat{x}_{0|-1}, P_{0|-1} \end{cases} \qquad \textbf{(6.3)}$$

With:

- $\hat{x}_{k|k}$ estimation of the state at time k based on k measurements.
- $\hat{x}_{k|k-1}$,$\hat{y}_{k|k-1}$ prediction of the state and of the output at time k based on k-1 measurements.
- $P_{k|k}$, $P_{k|k-1}$ matrix estimation and prediction at time k. It represents the covariance matrix of the state prediction error.
- V, W variance matrices of the process and measurement noise.
- C_k,A_k time-varying linearized matrices.

Now, let's move on with the implementation of the above strategy:

```
                %% LOCALIZATION
% Set the map for the localization problem:
nx=length(initial_pose);
localization_map=occupancyMap(~map);
% Define the process and measurement noises:
range=800;%maximum range of the rays (sufficient to cover all the map)
angles=linspace(0,2*pi-pi/10,10);%angles of the rays expressed w.r.t. robot orientation
measurement_noise=randn(length(input_commands),2*length(angles))*sqrt(5000);
for i=2:2:size(measurement_noise,2)
     measurement_noise(:,i)=deg2rad(measurement_noise(:,i));
end
process_noise=randn(length(input_commands),3)*sqrt(0.001);
process_noise(:,3)=deg2rad(process_noise(:,3));
```

The approach adopted in this work consists of identifying time-varying landmarks. These are computed by considering rays intersection between the robot and the surrounding around it. Sensor readings are useful to perform the measurements according to the model description. The measurement noise is chosen with high variance in order to simulate noisy measurements, on the contrary the process noise is small since the model description is accurate and so we want to trust it.

```
% Initialization of the Filter:
x_predicted=zeros(3,length(pose));
x_estimated=zeros(3,length(pose));
x_estimated(:,1)=initial_pose';
P_estimated=zeros(nx,nx,length(input_commands));
P_estimated(:,:,1)=10^-3*eye(nx);
P_predicted=zeros(nx,nx,length(input_commands));
% One-step ahead state prediction thanks to the eq. of the model:
x_predicted(:,1)=[x_estimated(1,1)+input_commands(1,1)*cos(x_estimated(3,1))*Ts ...
        ;x_estimated(2,1)+input_commands(1,1)*sin(x_estimated(3,1))*Ts...
        ;x_estimated(3,1)+input_commands(1,2)*Ts]+process_noise(1,:)';
% Linearized matrix A:
A=[1 0 -input_commands(1,1)*sin(initial_pose(3))*Ts;
   0 1 input_commands(1,1)*cos(initial_pose(3))*Ts;
   0 0 1];
% Covariance State prediction error thanks to EKF equations:
P_predicted(:,:,1)=A*P_estimated(:,:,1)*A'+cov(process_noise);
```

The above code is responsible for initializing the filter with the first estimation and prediction.

The entire evolution of the EKF is incorporated into the following loop:

```
i=2;
while i<=length(input_commands)
    % Generate points setting rays on the current pose (coming from the controller):
    intsectionPts =rayIntersection(localization_map,...
       [pose(i,1),localization_map.DataSize(1)-pose(i,2),...
       pose(i,3)],angles,range);

    % Compute the estimation:
    [x_estimated(:,i),P_estimated(:,:,i)]=KalmanEstimation(x_predicted(:,i-1),...
     P_predicted(:,:,i-1),nx,measurement_noise,intsectionPts,i,localization_map,pose(i,:));

    % Compute the prediction:
    [x_predicted(:,i),P_predicted(:,:,i)]=KalmanPrediction(x_estimated(:,i),...
     input_commands(i,:),Ts,process_noise,P_estimated(:,:,i),i);
i=i+1;
```

The simulation lasts until all the input commands computed in the controller framework are used. At each iteration, new sensor readings are performed for updating the current estimation and prediction. These are computed by the following functions:

Kalman Prediction:

```
function [x_predicted,P_predicted] = KalmanPrediction(x_estimated,input_commands,Ts,...
        process_noise,P_estimated,iteration)
    % Compute the predicted state according to the model:
    x_predicted=[x_estimated(1)+input_commands(1)*cos(x_estimated(3))*Ts ...
        ;x_estimated(2)+input_commands(1)*sin(x_estimated(3))*Ts...
        ;x_estimated(3)+input_commands(2)*Ts];
    % Add the process noise to the prediction:
    x_predicted=x_predicted+process_noise(iteration,:)';
    % Compute Linearized matrix A:
    A=[1 0 -input_commands(1)*sin(x_estimated(3))*Ts;
        0 1 input_commands(1)*cos(x_estimated(3))*Ts;
        0 0 1];
    P_predicted=A*P_estimated*A'+cov(process_noise);
end
```

It simply reflects the equation for the prediction using the inputs from the controller part.

Kalman Estimation:

```
function [x_estimated,P_estimated] = KalmanEstimation(x_predicted,...
     P_predicted,nx,measurement_noise,points,iteration,localization_map,real_pose)
    % Generate the linearized matrix C by computing the distances (linear and angular)
    % between the robot and the detected points:
    C=zeros(2*length(points),3);
    % Define the predicted output (linear and angular distance for each
    % points from the PREDICTED state
    d=zeros(2*length(points),1);
    % Define the real output (linear and angular distance for each
    % points from the REAL state)
    y=zeros(2*length(points),1);
   i=1;
   for j=1:length(points)
       % Derivative of the linear distance w.r.t. x coordinate:
       dz_dx=(points(j,1)-x_predicted(1))/sqrt((x_predicted(1)-points(j,1))^2+...
           (localization_map.DataSize(1)-x_predicted(2)-points(j,2))^2);
       % Derivative of the linear distance w.r.t. y coordinate:
       dz_dy=(points(j,2)-(localization_map.DataSize(1)-x_predicted(2)))/...
       sqrt((x_predicted(1)-points(j,1))^2+...
           (localization_map.DataSize(1)-x_predicted(2)-points(j,2))^2);
       C(i,:)=[dz_dx dz_dy 0];
       % Derivative of the angular distance w.r.t. x coordinate:
       dphi_dx=(points(j,2)-(localization_map.DataSize(1)-x_predicted(2)))/...
        ((x_predicted(1)-points(j,1))^2+...
        (localization_map.DataSize(1)-x_predicted(2)-points(j,2))^2);
```

```
% Derivative of the angular distance w.r.t. y coordinate:
dphi_dy=-(points(j,1)-x_predicted(1))/...
    ((x_predicted(1)-points(j,1))^2+...
    (localization_map.DataSize(1)-x_predicted(2)-points(j,2))^2);
C(i+1,:)=[dphi_dx dphi_dy -1];
% Predicted output: linear and angular distance of landmarks w.r.t. the predicted state
d_i=sqrt((x_predicted(1)-points(j,1))^2+...
    (localization_map.DataSize(1)-x_predicted(2)-points(j,2))^2);
d(i)=d_i;
d_angular=atan2((points(j,2)-(localization_map.DataSize(1)-x_predicted(2))),...
    (points(j,1)-x_predicted(1)))-x_predicted(3);
d(i+1)=d_angular;
% Real output: linear and angular distance of landmarks w.r.t. the real state:
y(i)=sqrt((real_pose(1)-points(j,1))^2+(localization_map.DataSize(1)-real_pose(2)...
    -points(j,2))^2);
y(i+1)=atan2((points(j,2)-(localization_map.DataSize(1)-real_pose(2))),...
    (points(j,1)-real_pose(1)))-real_pose(3);
        i=i+2;
    end
    % Add the noise to the real measurements:
    y=y+measurement_noise(iteration,:)';
    e=y-d; %innovation term
    S=C*P_predicted*C'+cov(measurement_noise);
    kalman_gain=P_predicted*C'*inv(S);
    x_estimated=x_predicted+kalman_gain*e;
    P_estimated=(eye(nx)-kalman_gain*C)*P_predicted;
end
```

At each call of the function, it generates the measurements with respect to the real state (real output) and to the predicted state (predicted output). Their difference is the innovation term of the filter. Computing the derivative terms, matrix C is populated and the estimation of the variables of interest is performed by the EKF equations.

Now, let's have a look at the results:

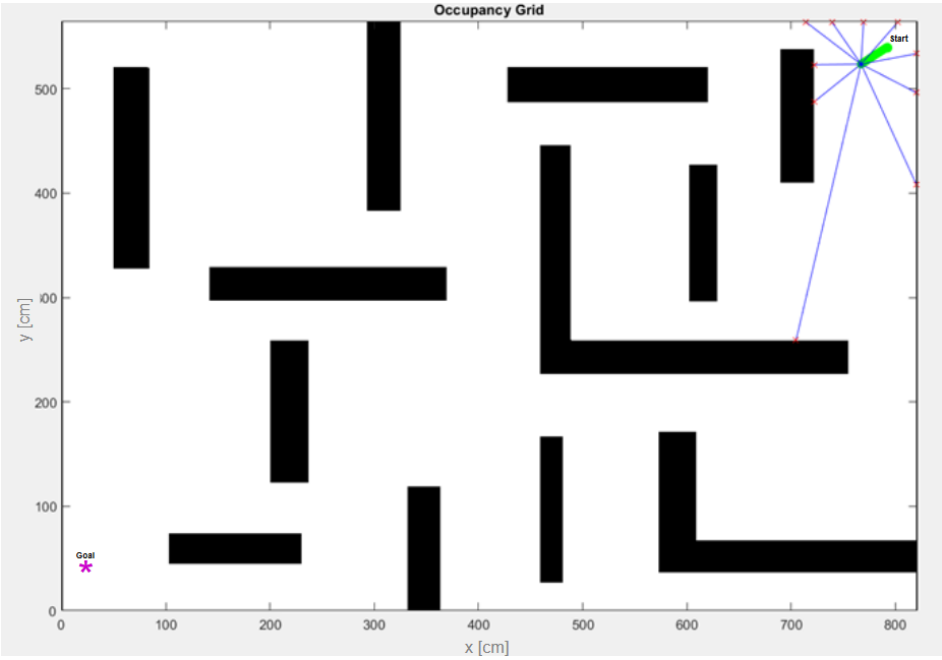

Fig. 6.1: Snapshot of the sensor reading evolution

The above figure illustrates a snapshot of the detecting operation performed by intersecting the rays with the local environment. Red points are detected and are used by the filter to obtain both prediction and estimation.

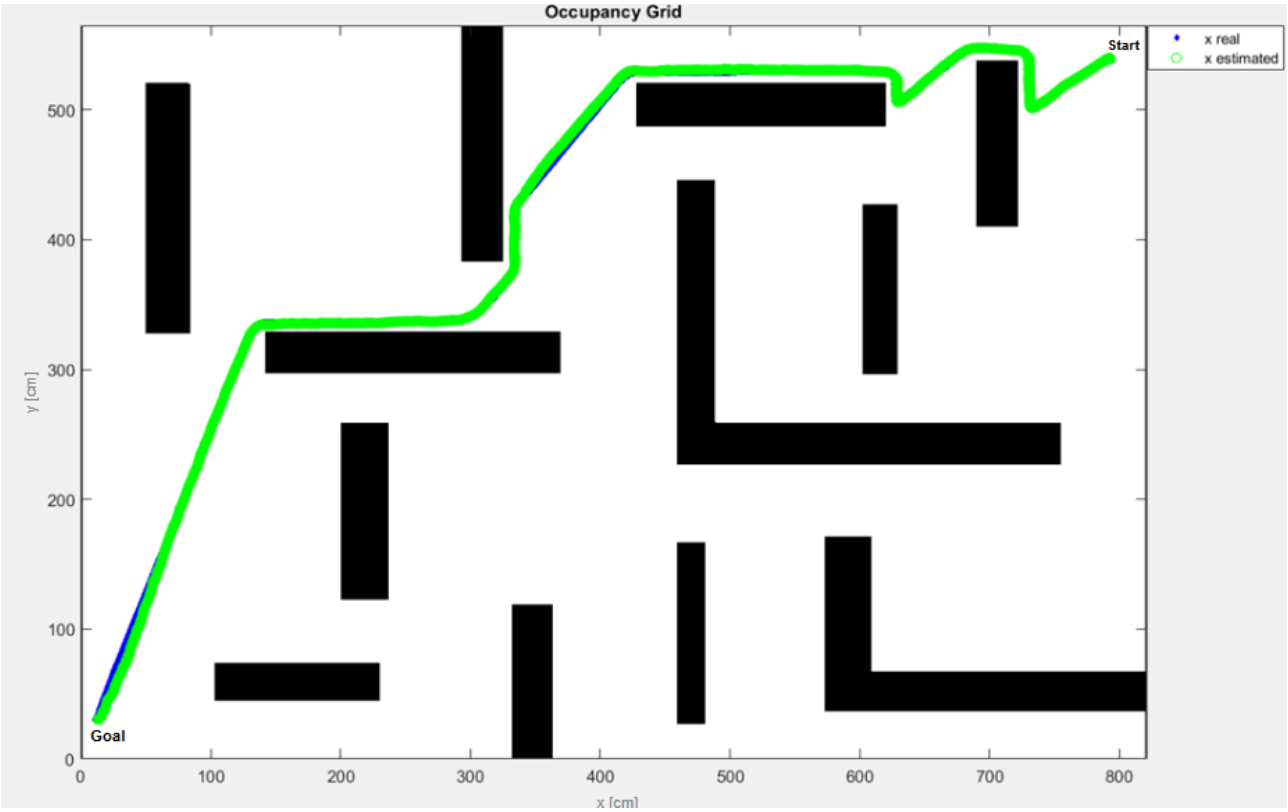

Fig. 6.2: Real and Estimated state

At the end of the simulation we can appreciate that the EKF generated a good estimation of the robot pose.